

Revisão de QA — Suíte E2E TOTVS Fluig Cassi

Documento de trabalho. Foi escrito para ser consultado durante a implementação dos pontos de melhoria — a Parte II é um checklist executável, com arquivo, linha e correção proposta em cada item.

Data da revisão	03/09/2026
Commit revisado	858f62b (branch emdash/teste-2-jxzx)
Base de medição	execução completa de 03/09/2026 — relatorios-2026-09-03/*.json
Critérios	skills playwright-test-creator, fluig-master, cassi-fluig-master
Papel	QA Senior Engineer — revisão de prontidão para entrega ao cliente

Veredito em uma frase

Projeto confiável e tecnicamente acima da média — a engenharia de teste está certa e a suíte de fato aumenta a qualidade do sistema. Mas **ainda não está pronto para entregar**, por quatro motivos que não são de mérito técnico e sim de *empacotamento*: o portão de CI está permanentemente vermelho, o alarme de defeito-corrigido grita falso toda execução, o repositório carrega 277 MB de evidência versionada, e a documentação-âncora está defasada em relação ao código. **São 2 a 3 dias de trabalho, não uma reescrita.**

Parte I — O diagnóstico

1. Método

Fonte	O que foi extraído
81 arquivos de spec / 233 testes	varredura das proibições absolutas da <code>playwright-test-creator</code>
41 Page Objects, 7 factories, 9 utils (~21.700 linhas)	arquitetura, isolamento, paralelismo
<code>relatorios-2026-09-03/*.json</code> (26 fatias)	placar recontado por status , não o declarado
<code>docs/</code> (13 documentos) + <code>README.md</code> + <code>CLAUDE.md</code>	rastreabilidade e drift documental
<code>.github/workflows/e2e.yml</code>	o gate que decide merge

Executados e limpos: `npm run typecheck ( tsc --noEmit , strict: true )` e `node --check` em 100% dos arquivos `.js / .mjs`.

2. Placar objetivo — números medidos, não declarados

Testes na execução completa (03/09/2026)	233 — 162 verdes, 71 vermelhos, 68,5 min
Cobertura do catálogo	156 de 187 casos (83%) , 31 lacunas todas com motivo declarado
Vermelhos sem veredito (timeout opaco)	0 — todos com mensagem de domínio ou <code>PRÉ-CONDIÇÃO AUSENTE</code>
<code>test.skip / test.fixme / test.only</code>	0
<code>console.log</code> no lugar de assertion	0
<code>waitForTimeout</code> como sincronização	1 — <code>pages/MedicaoContratoPage.js:274</code> , com 15 linhas de medição justificando
<code>force: true</code> abusivo	0 — as 3 ocorrências são comentários explicando por que <i>não</i> usar
<code>try/catch</code> engolindo assertion	0 — os 26 blocos são parse de JSON ou retry com rethrow
Locators	491 <code>getByRole</code> · 6 XPath (todos <code>ancestor:: relativos</code>) · zero <code>nth-child</code>
<code>// @ts-check</code>	100% dos arquivos

3. Conformidade com playwright-test-creator — Quality Gate item a item

Item do gate	Status	Evidência
Testes executados e analisados	OK	3 execuções completas documentadas (02 e 03/09), com JSON preservado
Sem skips injustificados	OK	zero skips no repositório inteiro
Sem flaky conhecido	OK	estudo de 735 execuções <code>--repeat-each=3</code> ; achou e corrigiu um falso verde real (CT-ADM-01-H)
Sem falha silenciosa	OK	verificado bloco a bloco
Assertion explícita em todo teste	OK	os 3 "sem expect" delegam a <code>expectPublicacaoBloqueada()</code>
Independência / paralelismo	OK	<code>fullyParallel: true</code> ; <code>utils/exclusividade.js</code> resolve o único recurso genuinamente único (staging de upload do Fluig)
Determinismo	RESSALVA	certificado do lado do teste; ver M-07 sobre a seed do faker
Massa fictícia (faker + sufixo único + prefixo QA)	OK	7 factories, CPF com DV válido, domínio <code>example.test</code> , nada hardcoded
Evidência de falha no relatório	OK	fixture <code>evidence { auto: true }</code> anexa screenshot, URL, <code>titlePath</code> , erro e comando de reprodução — e omite screenshot em spec de API para não fabricar PNG branco que <i>parece</i> evidência
Segredos só em variável de ambiente	OK	<code>.env.test</code> nunca esteve no histórico (verificado com <code>git log --all</code>)
Falha real gera FAIL — validado	OK	os 71 vermelhos são a prova viva
Código verificado + lint	OK	<code>tsc --noEmit</code> limpo com <code>strict: true</code>
Roda local e em CI	RESSALVA	roda — mas o gate de CI não é utilizável hoje (M-01)

14 de 17 plenos, 3 com ressalva. Nenhuma violação das proibições absolutas.

4. Conformidade com cassi-fluig-master — o conhecimento do cliente

O projeto absorveu corretamente os **sete fatos técnicos** da skill:

- `page.evaluate` + `fetch` em vez de `page.request` — o WAF barra por falta de `User-Agent` e `Referer` (aplicado em `utils/cancelamento-fluig.js` e em 8 specs).
- D-01 isolado até o `targetState: 6` — o teste afirma a **causa**, não o sintoma.

- `cancelInstances` em lote com `cancelText` obrigatório, no `globalTeardown`, com corte por `process.uptime()`.
- Interceptação de dataset **pelo corpo** (`postData()`), porque no Fluig todo dataset usa a mesma URL — `utils/dataset-fluig.js`.
- "Interceptar muda o comportamento da aplicação": a trava antiduplo-clique é testada **segurando a requisição em voo**, não abortando.
- **PRÉ-CONDIÇÃO AUSENTE** distinguindo instabilidade do Protheus de defeito — e a execução de 03/09 **descartou uma janela degradada inteira** e reexecutou 6 fatias conforme o protocolo. Isso é disciplina de medição de nível sênior.

Onde há lacuna (detalhe em **M-06**): dos *seis fatos de negócio* da skill — que são a fonte declarada das assertions — a **regra de concorrência numérica** e a **trava anti-bypass da alçada** não têm caso no catálogo nem teste.

Regra (DOCUMENTADO pelo cliente)	Caso no catálogo	Teste
Sem dispensa: mínimo 3 fornecedores · Com dispensa: exatamente 1	ausente	ausente
Alçada: trava rígida contra bypass por "inspecionar elemento" (Auditoria Interna)	ausente	ausente
Arredondamento fiscal / <i>saving</i> são do Protheus	ausente	ausente
Alçada: consenso de 100%	CT-CMP-05	<code>tests/e2e/portais/alcadas-orcamentaria.spec.js:74</code>

5. Por que este projeto merece confiança

Cinco pontos que quase nenhuma suíte tem:

1. A suíte prova o que afirma, inclusive quando afirma "não fez". `utils/guarda-criacao.js` foi **invertido** depois de um incidente: bloqueia toda escrita no host *exceto* uma allowlist de leitura. A versão anterior interceptava só `/process-management/**`, e os formulários avulsos escapavam por `workflowView/send` — resultado: testes que faziam `expect(guarda.tentativas()).toBe(0)` e **passavam sem provar nada**. Encontrar isso e inverter a lógica é o comportamento correto; documentar o incidente no cabeçalho do arquivo é o comportamento de quem quer que o time não repita.

2. Provas negativas sem escrever na base. `utils/captura-payload.js` intercepta o `POST .../start`, lê os ~101 campos e **aborta**. Defeitos que antes só apareciam na SC já criada (D-01, D-02, D-04, incoerência de contrato) viraram testes não-destrutivos. É a técnica de maior alavancagem do projeto.

3. A massa não tem ponto único de falha. `utils/massa-contratos.js` usa *rendezvous hashing* (FNV-1a) entre a identidade do teste e o número do contrato, distribuindo entre os 554 vigentes, com **lock de diretório** por contrato e devolução na fixture `evidence` — inclusive quando o teste morre por timeout. E há um detalhe que só quem mediu descobre: as tags são

removidas do título antes do hash, porque marcar um teste com `@bug` trocava o contrato sorteado e dois resultados deixavam de ser comparáveis.

4. O livro-razão escuta a rede, não a convenção. `fixtures/fixtures.js` registrava a massa lendo anotações `*-criada`. O filtro estava incompleto e deixou **11 solicitações órfãs**. A correção passou a escutar a *resposta de criação* — e não filtra por `resposta.ok()`, porque foi medido que `prc_questionario_v2` cria a instância **respondendo HTTP 500**. Engenharia baseada em medição, não em suposição.

5. Os vermelhos são um produto, não um problema. 71 vermelhos, **0 sem veredito**. A convenção `@bug` (vermelho intencional) contra `@achado` (polaridade invertida — verde hoje, vermelho quando o comportamento mudar) é conceitualmente correta e rara. Os 8 `@achado` seguem verdes; os 5 processos de RH que abrem sem bloqueio de grupo estão versionados como `achado` em vez de depender de alguém lembrar.

Parte II — Checklist de implementação

Bloqueadores para a entrega

M-01 · O portão de CI está vermelho e vai continuar vermelho

Onde: `.github/workflows/e2e.yml`, `job regressao` **Comando do gate:** `PULAR_DESTRUTIVOS=1 npx playwright test --grep-invert "@bug|@achado"`

Escopo recontado na execução de 03/09: **136 testes, 9 falhando.**

Spec / linha	Caso	Natureza
<code>tests/e2e/acompanhamento-contratos/criacao-solicitacao.spec.js:513</code>	CT-ACC-06-S2	defeito de produto, sem @bug
<code>tests/e2e/acompanhamento-contratos/modal-solicitacao-compra.spec.js:50</code>	—	sem tag, sem triagem
<code>tests/e2e/acompanhamento-contratos/payload-solicitacao.spec.js:206</code>	D-02	defeito catalogado, sem @bug
<code>tests/e2e/compras/ciclo-cotacao.spec.js:168</code>	CT-COT	PRÉ-CONDIÇÃO AUSENTE estrutural
<code>tests/e2e/compras/negociacao-proposta.spec.js:131</code>	CT-NEG	PRÉ-CONDIÇÃO AUSENTE estrutural
<code>tests/e2e/contratos/validacoes-faturamento.spec.js:79</code>	CT-FAT-02-S2	defeito de produto, sem @bug
<code>tests/e2e/contratos/validacoes-faturamento.spec.js:254</code>	CT-FAT-02-S3	inalcançável pela conta da automação
<code>tests/e2e/plataforma/catalogo-invariante.spec.js:149</code>	CT-PLT-10-H	vermelho assumido no <code>CLAUDE.md</code>
<code>tests/e2e/plataforma/catalogo-invariante.spec.js:224</code>	CT-PLT-10-H	idem

Por que é bloqueador. O comentário do próprio workflow diz: *"um gate que nunca fica verde deixa de informar qualquer coisa — e o time passa a mergear por cima dele, que foi o que aconteceu no PR #2"*. **O gate reintroduziu exatamente o problema que foi criado para resolver.**

Correção proposta

- ☐ Aplicar `@bug` aos 3 defeitos já catalogados (`CT-ACC-06-S2`, `D-02` em `payload-solicitacao.spec.js:206`, `CT-FAT-02-S2`).
- ☐ Triar `modal-solicitacao-compra.spec.js:50` — decidir se é defeito (`@bug`) ou comportamento real a registrar (`@achado`).

- ☐ Criar a tag `@pre-condicao` para `CT-COT`, `CT-NEG` e `CT-FAT-02-S3`, e excluí-la do gate: `--grep-invert "@bug|@achado|@pre-condicao"`. É o item 3 pendente de `docs/estado-do-gate.md` ("portão de pré-condição por execução").
- ☐ Decidir explicitamente sobre `catalogo-invariante.spec.js`: atualizar o inventário versionado, ou tirá-lo do gate declarando que ele é invariante informativo.
- ☐ **Meta de aceite:** gate verde, **ou** o número exato de vermelhos esperados declarado no README e conferido por um passo do workflow.

M-02 · O alarme de "defeito corrigido" grita falso toda execução

Onde: `tests/e2e/plataforma/erros-de-console.spec.js:152` · `scripts/alerta-bug-corrigido.mjs`

Na execução de 03/09, **8 testes @bug passaram** — 7 deles são o mesmo teste parametrizado:

```
erros-de-console.spec.js:152 — 8 rotas, TODAS marcadas @bug
VERDE  Home · pageprocessstart · pagecentraltask · acompanhamentoContrato
VERDE  gerenciaCompras · PORTAL_TRACKER_COMPRAS_CONTRATOS · ecmnavigation
VERM.  Portal do Comprador <-- esta, e S0 esta, e o defeito
```

A tag foi aplicada no template do `for`, então 7 rotas carregam um `@bug` que não merecem. `scripts/alerta-bug-corrigido.mjs` sai com código 1 e emite 7 `::warning::` a cada run. **Um alarme que sempre dispara é um alarme que ninguém lê.**

Correção proposta

- ☐ Tag condicional por rota — ex.: incluir `@bug` apenas quando `rota === '/portal/p/1/portal-do-comprador'`.
- ☐ `tests/e2e/portais/atribuicao-comprador.spec.js:36` — o teste afirma o comportamento **real** ("a aba Atribuir *não* lista SCs") e está verde. Pela tabela do próprio `CLAUDE.md`, isso é `@achado`, não `@bug`. Reclassificar.
- ☐ **Aceite:** `node scripts/alerta-bug-corrigido.mjs` sai 0 numa execução normal.

M-03 · 277 MB de evidência versionada, com duplicatas

```
36M  relatorio-falhas-2026-09-02.zip
36M  relatorio-2026-09-02/relatorio-falhas-2026-09-02.zip <-- o MESMO arquivo
27M  relatorio-falhas-2026-09-03.zip
17M  relatorio-falhas-2026-09-03.html      17M  relatorios-2026-09-03/base.html
15M  relatorio-falhas-2026-09-02.html      15M  relatorios/base.html
14M  relatorio-falhas.html                 8.1M  relatorios-2026-09-03/merged.json
```

O `.gitignore` acerta em `test-results/` e `playwright-report/`, mas os relatórios entraram mesmo assim. Cada dia de execução acrescenta **~60 MB permanentes** ao histórico. Entregar isso ao cliente é entregar um `git clone` de 277 MB que só cresce.

Correção proposta

- ☐ Mover `.zip`, `.pdf` e `.html` de relatório para *release asset* ou artefato de CI — o workflow **já faz** `upload-artifact` com 30 dias de retenção.
- ☐ Manter versionado apenas o `.md` da análise (241 KB) e os JSON de status, se necessários para auditoria.
- ☐ Acrescentar ao `.gitignore`: `relatorio-falhas*.html`, `relatorio-falhas*.zip`, `relatorio-falhas*.pdf`, `relatorios*/base.html`, `relatorios*/merged.json`.
- ☐ Limpar o histórico (`git filter-repo` ou branch órfã) **antes** de entregar o repositório.

M-04 · `probe.mjs` versionado com caminho local de máquina

Onde: `probe.mjs:2`

```
const OUT='/home/dev1/.claude/jobs/739913d6/tmp';
```

Não roda em nenhuma outra máquina, duplica `scripts/sonda-grade.mjs` e tem `catch{}` vazio. O `.gitignore` tem `probe-*.mjs`, que **não casa** com `probe.mjs` — a regra ficou específica demais, exatamente como no caso do symlink `node_modules` já corrigido em 03/09.

Correção proposta

- ☐ `git rm probe.mjs` — a função já existe em `scripts/sonda-grade.mjs`.
- ☐ Trocar a regra do `.gitignore` de `probe-*.mjs` para `probe*.mjs`.

Importantes — corrigir antes de entregar, não bloqueiam

M-05 · A documentação-âncora está defasada

`CLAUDE.md` declara `docs/estado-do-gate.md` como "fonte da verdade" das medições. Ele diz:

	Documento	Real hoje
Specs	65	81
Testes	177	233
Page Objects	36	41
Última medição	25-26/08	há duas execuções completas depois

E há três números divergentes sobre a mesma coisa:

Documento	Casos no catálogo	Cobertos	Sem teste
<code>CLAUDE.md</code> : 292	163	132	31
<code>docs/cobertura.md</code> (gerado)	187	156	31
<code>README.md</code> : 408	—	—	30

Além disso, a pergunta 2 de "Perguntas em aberto para a Cassi" (README.md) descreve um problema **já resolvido** nesta versão (o default de Renovação Contratual na factory).

Isso importa: é o primeiro documento que o cliente lê, e ele contradiz a suíte.

Correção proposta

- ☐ Regenerar docs/estado-do-gate.md com a execução de 03/09 (233 / 162 / 71).
- ☐ Alinhar CLAUDE.md:292 e README.md:408 ao número **gerado** por npm run cobertura — nunca digitado à mão.
- ☐ Remover ou reescrever a pergunta 2 de "Perguntas em aberto para a Cassi".

M-06 · Ponto cego de regra de negócio, não declarado como lacuna

As 31 lacunas versionadas têm **todas** motivo declarado — disciplina exemplar. Mas as regras da tabela da seção 4 **não são lacunas: são ausências**, porque nem no catálogo estão.

A causa provável é legítima (a fila de Cotações está vazia por cascata do D-01), só que essa causa **não está registrada em lugar nenhum**. Um caso de catálogo com bloqueado por D-01 custa 10 minutos e transforma um ponto cego em pendência rastreada.

Correção proposta

- ☐ Criar em docs/catalogo-casos.md : CT-COT-03 — concorrência mínima (3 fornecedores sem dispensa / exatamente 1 com dispensa), com motivo bloqueado por D-01: a fila de Controle de Cotações nunca recebe registro.
- ☐ Criar CT-CMP-05-S2 — bypass client-side da alçada (exigência da Auditoria Interna), com o motivo de bloqueio medido.
- ☐ Declarar o motivo de arredondamento fiscal / saving ficarem fora (regra do Protheus, fora do escopo do Fluig — se for esse o caso, registrar assim).
- ☐ Rodar npm run cobertura e confirmar que o script aceita os novos motivos.

M-07 · A reprodução por FAKER_SEED é parcial, e o relatório promete que é exata

Onde: fixtures/fixtures.js:24

```
faker.seed(FAKER_SEED); // executa no CARREGAMENTO do modulo
```

Cada worker é um processo, então todos partem da mesma seed e consomem a sequência **na ordem em que o runner despachou os testes**. Com --workers=4 , reexecutar com a seed do relatório reproduz o mesmo *conjunto* de valores, mas **não garante o mesmo valor para o mesmo teste**.

A fixture evidence anexa reproduzirCom: FAKER_SEED=<v> npx playwright test em toda falha — a promessa é de reprodução exata.

Correção proposta (uma linha, reaproveitando o hash32 que já existe em utils/massa-contratos.js)

☐ Semear **por teste**, dentro da fixture `evidence : faker.seed(FAKER_SEED ^ hash32(testInfo.titlePath.join('|')))`.

☐ Alternativa barata, se preferir não mexer: documentar que a reprodução exata exige `--workers=1`, e ajustar a string `reproduzirCom` para incluí-lo.

M-08 · O job noturno de destrutivos triplica a massa em cada falha

Onde: `playwright.config.js` (`retries: process.env.CI ? 2 : 0`) e `.github/workflows/e2e.yml`, job `destrutivos`.

O job roda `npx playwright test --grep @destrutivo --workers=1` **sem sobrescrever** `retries`. O comentário do próprio workflow dá esse motivo para não rodar destrutivos em PR — *"com retries: 2, cada falha vira três solicitações"* — mas o agendado herda a mesma configuração. Com os 26 destrutivos vermelhos de 03/09, isso é **até 78 registros por noite** na base de homologação, em vez de 26.

Correção proposta

☐ Acrescentar `--retries=0` ao comando do job `destrutivos`.

M-09 · Sete IDs contados "por menção em prosa"

O próprio `docs/cobertura.md` avisa que estes foram casados fora de título e são candidatos a falso positivo nos 156 cobertos:

`CT-CMP-02-S2` · `CT-CMP-02-S4` · `CT-DEP-01-H` · `CT-DEP-02-S1` · `CT-PLT-09-S1` · `CT-PLT-10-H` · `CT-SUB-01-H`

Correção proposta

☐ Para cada um: confirmar que existe teste e **levar o ID para o título do teste**; ou declarar como lacuna com motivo.

☐ Regenerar `npm run cobertura` e confirmar que o aviso some.

Recomendação de valor — não é defeito, é entregável a mais

M-10 · Devolver ao time de desenvolvimento a lista de âncoras ausentes

`docs/mapa-do-ambiente.md:177` já cataloga que os ícones da coluna "Ação" são âncoras vazias sem `aria-label` e que o seletor de idioma é `<img>` sem `alt`. A suíte contorna com o atributo `title` e com CSS — funciona, mas é dívida técnica do produto, não do teste.

A skill `playwright-test-creator` manda **recomendar** `data-testid` **ao time de desenvolvimento** em vez de criar seletor frágil, e **não há nenhuma menção a** `data-testid` **no repositório**.

Correção proposta

- ☐ Criar `docs/recomendacoes-de-testabilidade.md` : os pontos onde a testabilidade custa caro (os ~40 locators CSS crus em `pages/`), e o atributo que resolveria cada um.
- ☐ Entregar junto com o relatório de falhas — soma **acessibilidade + testabilidade** e é o tipo de item que faz a inspeção valer além dos 71 vermelhos.

Ordem de execução sugerida

Ordem	Item	Esforço	Bloqueia entrega?
1	M-01 — gate de CI verde ou com vermelhos declarados	~4 h	sim
2	M-02 — tag condicional em <code>erros-de-console</code> ; reclassificar <code>atribuicao-comprador</code>	~30 min	sim
3	M-03 — tirar 277 MB de evidência do versionamento	~1 h	sim
4	M-04 — remover <code>probe.mjs</code> , corrigir o <code>.gitignore</code>	~5 min	sim
5	M-05 — regenerar <code>estado-do-gate.md</code> , alinhar <code>CLAUDE.md</code> e <code>README</code>	~2 h	sim
6	M-08 — <code>--retries=0</code> no job noturno	~1 min	não
7	M-07 — semear o faker por teste	~20 min	não
8	M-06 — declarar a lacuna de concorrência de fornecedores	~1 h	não
9	M-09 — resolver os 7 IDs por menção em prosa	~1 h	não
10	M-10 — documento de recomendações de testabilidade	~2 h	não

Feitos os itens 1 a 5, a entrega está assinável. Os itens 6 a 9 podem ir na entrega seguinte, desde que declarados. O item 10 é o diferencial a oferecer ao cliente.

Conclusão

A suíte cumpre o propósito. Ela não existe para gerar testes — existe para dar confiança sobre o comportamento do produto, e dá: 71 falhas, todas com veredito, **18 delas defeitos de produto que ninguém tinha achado antes** (fail-open no formulário clássico, GED sem allowlist de extensão, isolamento horizontal na API de processos, telemetria LGPD, SC criada sem anexo obrigatório *no servidor*). O ganho de qualidade não é hipotético — está catalogado e reproduzível.

A engenharia de teste é sólida e não precisa de retrabalho. Não foi encontrado falso verde, falha silenciosa, skip escondido ou teste ajustado para passar. Foi encontrado o contrário: um projeto que achou o próprio falso verde, mediu, corrigiu e documentou o incidente para o time não repetir.

O que falta é empacotamento — e é rápido.